WhoPlay — Guia 3: Jogos e check-in

Oct 28, 2026 · @Sebastião Cardoso

Responsável sugerido: Hélio José. Este guia cobre tudo o que tem a ver com jogos: criar um jogo, vê-lo no mapa, entrar e mostrar o QR Code do check-in. O código está no guia principal, "WhoPlay — Guia da base de dados e da API"; aqui fica o que te cabe, por que ordem e como sabes que acabaste.

Guia principal (todo o código): WhoPlay — Guia da base de dados e da API

O que vais construir

No fim deste guia, quem carrega no botão + cria um jogo que fica gravado na base de dados. O jogo aparece no Mapa, o Detalhe mostra quem vai e quantas vagas restam, e entrar no jogo dá um QR Code único para o check-in.

Camada	O que é teu
Ecrãs	07 Criar jogo, 08 Mapa (cartão e marcador do jogo), 09 Detalhe do jogo, 10 Check-in
Tabelas	`jogos`, `jogo_participantes`
Rotas da API	`GET /jogos`, `GET /jogos/:id`, `POST /jogos`, `POST /jogos/:id/entrar`, `GET /jogos/:id/checkin`, `POST /jogos/checkin`
Pontos técnicos	Transações, `SELECT ... FOR UPDATE` (a última vaga), UUID para o QR, vagas calculadas e não guardadas

Ficheiros

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`database/schema.sql` (secções 5 e 6)	Juntar	Parte 3
`database/seed.sql` (jogo de Padel e participantes)	Juntar	Parte 4
`database/consultas.sql` (consultas 5 e 10)	Juntar	Parte 5

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`backend/src/routes/jogos.js`	Criar	Parte 10.1
`backend/src/server.js`	Tirar o `//` da linha `/jogos`	Parte 6.8
`lib/data/models.dart` (`Participante`, `Jogo`)	Juntar debaixo de `// Guia 3`	Parte 12.5
`lib/services/api.dart` (`jogos`, `jogo`, `criarJogo`, `entrarNoJogo`)	Juntar debaixo do marcador "jogos (Guia 3)"	Parte 12.4
`lib/screens/create_game_screen.dart`	Alterar	Parte 12.15
`lib/screens/game_detail_screen.dart`	Alterar	Parte 12.16
`lib/screens/map_screen.dart`	Alterar (4 mudanças)	Parte 12.17
`lib/screens/checkin_screen.dart`	Alterar (5 mudanças)	Parte 12.18

Quem faz o quê na equipa

Guia	Responsável sugerido	Funcionalidade
1	Sebastião	Contas e perfil + base comum
2	Tchali	Jogadores e compatibilidade
3	Hélio	Jogos e check-in
4	Cristiano	Convites, avisos e integração final

Antes de começar

- A **Fase 0 do Guia 1** tem de estar no `main` (servidor, autenticação, tabelas `utilizadores` e `modalidades`, `api.dart` base). As tuas tabelas têm chaves estrangeiras para essas 2.
- XAMPP, Node.js e VS Code com Thunder Client, como no guia principal.

```
git checkout main
git pull
git checkout -b feature/jogos
cd backend && npm install && cp .env.example .env
```

Corre o `schema.sql` e o `seed.sql` que vieram no `pull`.

Passo a passo

1. Base de dados

- [ ] No fim do `schema.sql`, juntar as secções **5 (`jogos`)** e **6 (`jogo_participantes`)** da Parte 3.
- [ ] No fim do `seed.sql`, juntar o bloco "Jogo de Padel no próximo sábado" e o `INSERT INTO jogo_participantes` (Parte 4).
- [ ] Correr os 2 scripts e confirmar no phpMyAdmin: 1 jogo e 3 participantes, cada um com um `codigo_checkin` diferente.
- [ ] No `consultas.sql`, juntar as consultas **5** (vagas livres) e **10** (entrar num jogo), e o `DELETE` que desfaz a 10 (Parte 5). Correr a 5: Padel Club X com 1 vaga.

2. API

- [ ] `backend/src/routes/jogos.js` (Parte 10.1). Ele usa o `distanciaKm` de `utils/compatibilidade.js`, que é do Guia 2. Se esse ficheiro ainda não estiver no `main`, copia-o **exatamente igual** da Parte 9.1: um ficheiro idêntico nos 2 ramos não dá conflito no Git.
- [ ] O `POST /jogos/:id/entrar` grava um aviso para o organizador na tabela `notificacoes`, que é do Guia 4. Se ela ainda não existir, essa rota dá erro 500. Até o Guia 4 entregar, põe `//` nas 4 linhas do `INSERT INTO notificacoes` dessa rota e escreve `// TODO: descomentar quando o Guia 4 entrar`.
- [ ] No `server.js`, tirar o `//` da linha `app.use('/jogos', ...)`.
- [ ] Testar no Thunder Client os pedidos 8, 9 e 15 da Parte 11.3 e os passos 4 a 6 da Parte 11.4.

3. App

- [ ] No `models.dart`, debaixo de `// Guia 3`, juntar `Participante` e `Jogo` (Parte 12.5).
- [ ] No `api.dart`, debaixo do marcador "jogos (Guia 3)", juntar `jogos`, `jogo`, `criarJogo` e `entrarNoJogo` (Parte 12.4).
- [ ] `create_game_screen.dart` (Parte 12.15).

- [ ] `game_detail_screen.dart` (Parte 12.16), incluindo a `_ParticipantRow` nova e a `_ExternalRow`.
- [ ] `checkin_screen.dart`: as 5 mudanças da Parte 12.18.
- [ ] `map_screen.dart`: as 4 mudanças da Parte 12.17.
- [ ] `flutter analyze` sem problemas.

O `GameDetailScreen` passa a precisar de `jogoId` e o `CheckInScreen` de `jogo` e `codigo`. Se o `flutter analyze` acusar algum sítio que ainda os chama sem esses parâmetros, é código antigo: quem abre estes ecrãs são o Mapa, o Criar jogo e o Detalhe, que são todos teus.

Ligações com os colegas

Recebes	De quem	Enquanto não chega
Servidor, autenticação, `utilizadores`, `modalidades`	Guia 1 (Fase 0)	Não dá para começar
`distanciaKm` em `utils/compatibilidade.js`	Guia 2	Copia o ficheiro igual (passo 5)
Tabela `notificacoes`	Guia 4	Comenta o aviso ao organizador (passo 6)

Entregas	A quem
Tabela `jogos` e a rota `GET /jogos`	Guia 4, para o aviso "Falta 1 jogador para o teu jogo" do `seed.sql` fazer sentido
`MapScreen` e `CreateGameScreen`	Guia 4, que acrescenta a `key` do `MapScreen` no `home_shell.dart`

Conflito esperado: se o Guia 4 entrar no `main` antes de ti, o Git pode mostrar um conflito no fim do `schema.sql` e do `seed.sql`, porque os 2 acrescentaram linhas no mesmo sítio. Resolve mantendo as 2 partes: as tuas secções 5 e 6 primeiro, depois as 7 e 8 do Guia 4. A ordem entre elas não importa para o MySQL, porque nenhuma tabela de um aponta para as do outro.

Testes de aceitação e entrega

Com a base de dados acabada de recarregar:

- [ ] `GET /jogos` (token do Sebastião) → 1 jogo, Padel Club X, `vagas: 1`, `distancia_km: 2.4`
- [ ] `GET /jogos/1` → participantes Sebastião (organizador), Helio e Cristiano, por esta ordem; `eu_participo: true`
- [ ] `POST /jogos` com `vagas_abertas: 4` e `capacidade: 4` → 400 "Número de vagas inválido para esta capacidade."
- [ ] Um utilizador novo faz `POST /jogos/1/entrar` → 201 com `codigo_checkin`; repetir devolve o mesmo código
- [ ] O Tchali tenta entrar a seguir → 409 "Já não há vagas neste jogo."
- [ ] `POST /jogos/checkin` com esse código: pelo utilizador novo → 403; pelo Sebastião → 200
- [ ] Na app, Mapa → Ver jogo mostra o jogo de sábado com 3 de 4 e 1 vaga livre
- [ ] Na app, Ver o meu QR Code mostra o QR com a data e o local do jogo
- [ ] Na app, botão + → Criar jogo abre o detalhe do jogo novo, com o Sebastião como organizador
- [ ] `flutter analyze` sem problemas

Entrega:

```
git add database backend/src lib
git commit -m "Jogos e check-in: BD, API e ecrãs"
git push -u origin feature/jogos
```

Abre o Pull Request de `feature/jogos` para `main`. Se tiveres comentado o aviso ao organizador (passo 6), diz isso na descrição do Pull Request, para o Guia 4 se lembrar de o descomentar.